More insights on encryption + authentication

Encryption vs Integrity

- **Integrity**
  - *Prevents message spoofing (injection)*
  - *Prevents message tampering (modification)*

- **Integrity may be the only requirement**
  - *Goal: everybody should "see"*
  - *But nobody should be able to change…*

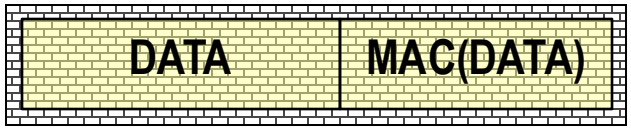
- **Encryption may NOT guarantee integrity!**
  - *We have already discussed this quite a lot!!*

Encrytion does not nearly guarantee integrity!

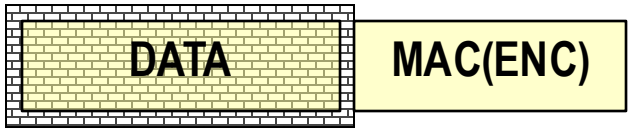
- **One time pad (one time random key K): perfect secrecy… But:**
  - *Encryption: ENC(M) → C = M⊕K*
  - *But:*
    - →C⊕M' = (M⊕K)⊕M' = (M⊕M')⊕K=ENC(M⊕M')
    - →"pay 1000 $" ⊕ "…(1 ⊕ 9)…" → "pay 9000 $"
- **Not the only case**
  - *RC4: same as above*
  - *Homomorphic encryption: modifiable by design!*
    - →including RSA
- **In general, don't trust encryption mechanisms for integrity**
  - *unless they are explicitly designed ALSO for it (authenticated encryption, e.g. AES-CCM, AES-GCM, etc)*
  - *TLS 1.3:* **ONLY** *(!!) AEAD ciphersuites!!*
    - →AEAD = Authenticated Encryption with Associated Data see e.g. IETF RFC 5116

How to combine ENC + MAC?

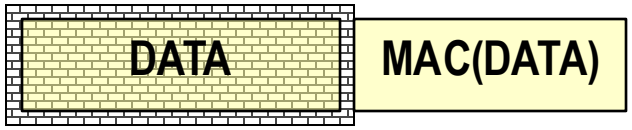
TLS: MAC then ENCRYPT



IPsec: ENCRYPT then MAC



SSH: ENCRYPT and MAC



Issue: which construction is best, in the assumption of

- *GENERAL (semantically secure) Encryption scheme*
- *GENERAL (unforgeable) MAC scheme*

Encrypt and MAC (SSH)

➔**Most insecure!**

- ⇨*Unforgeable MAC construction does NOT guarantee anything about information leakage!*
- ⇨*MAC (not encrypted, and applied to plaintext) may reveal something about the message!*

➔**Why? ☺**

MAC then Encrypt (TLS)

- **Not recommended!**
- **Pathological counterexamples**
  - Krawczyk, Crypto 2001: explicit costruction, active attack against "strange" (but shannon' secure) ENC
- **Practical attacks found**
  - Degrabriele, Paterson, ACM CCS 2010, on some unusual (but legitimate) IPsec configurations
    - AH then ESP with encryption only
- **Opens the door to Chosed Ciphertext Attacks**
  - *More later: Padding oracle, lucky 13, etc*

Encrypt then MAC

➔**Provably secure**

⇨*If semantically secure Encryption scheme*

➔Note: ENC not required to be secure against active attacks!

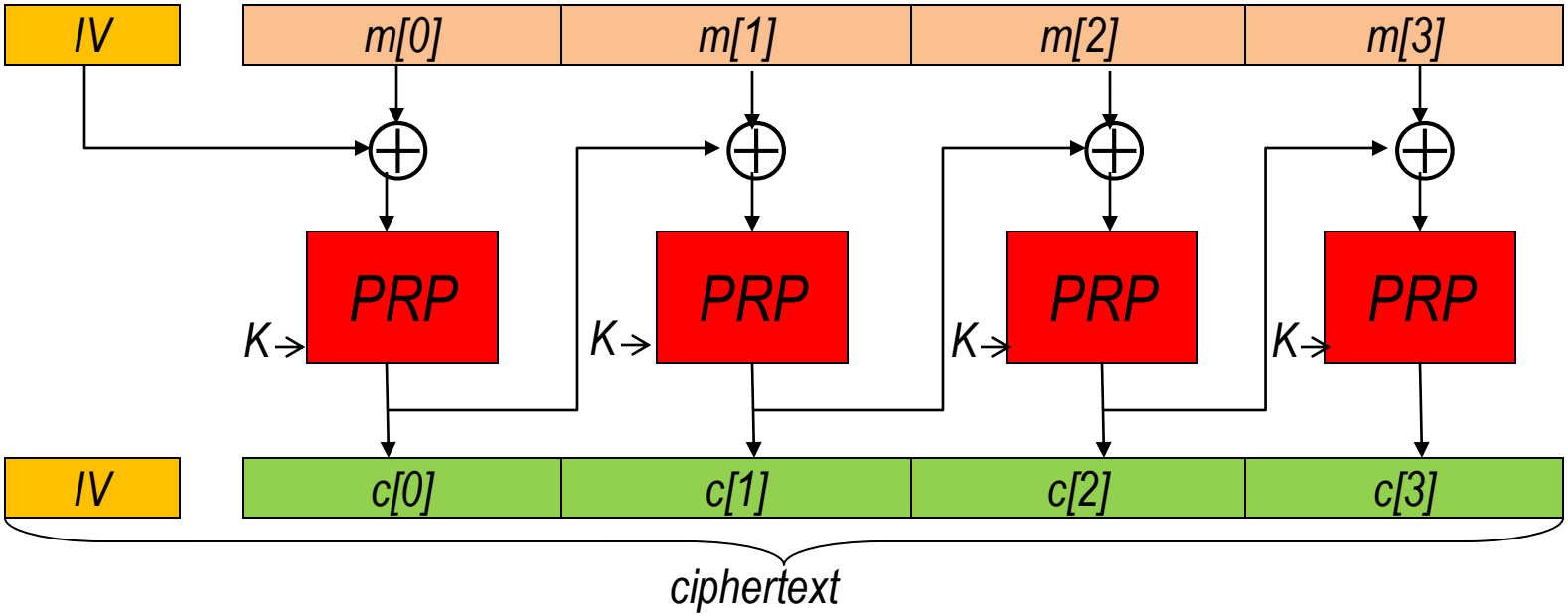
⇨*And unforgeable MAC scheme*

➔**Conclusion:**

⇨*If given the choice,*
ALWAYS use ENC then MAC, i.e. IPsec-style

⇨**EVEN BETTER: avoid the problem with AEAD!**

Recap: CBC encryption



➔**First block**

⇨*XOR message m[0] with random initialization vector IV*

➔**Subsequent blocks**

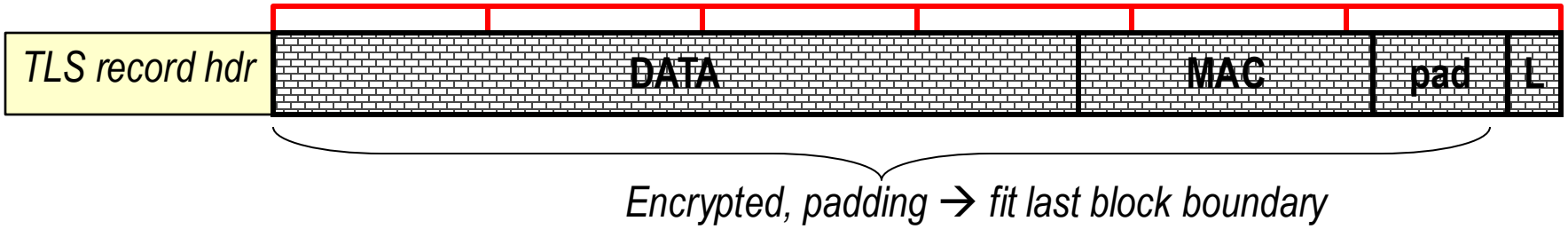
⇨*XOR message m[i] with previous ciphertext block c[i-1]*

MAC-then-encrypt in TLS: vulnerable with CBC!

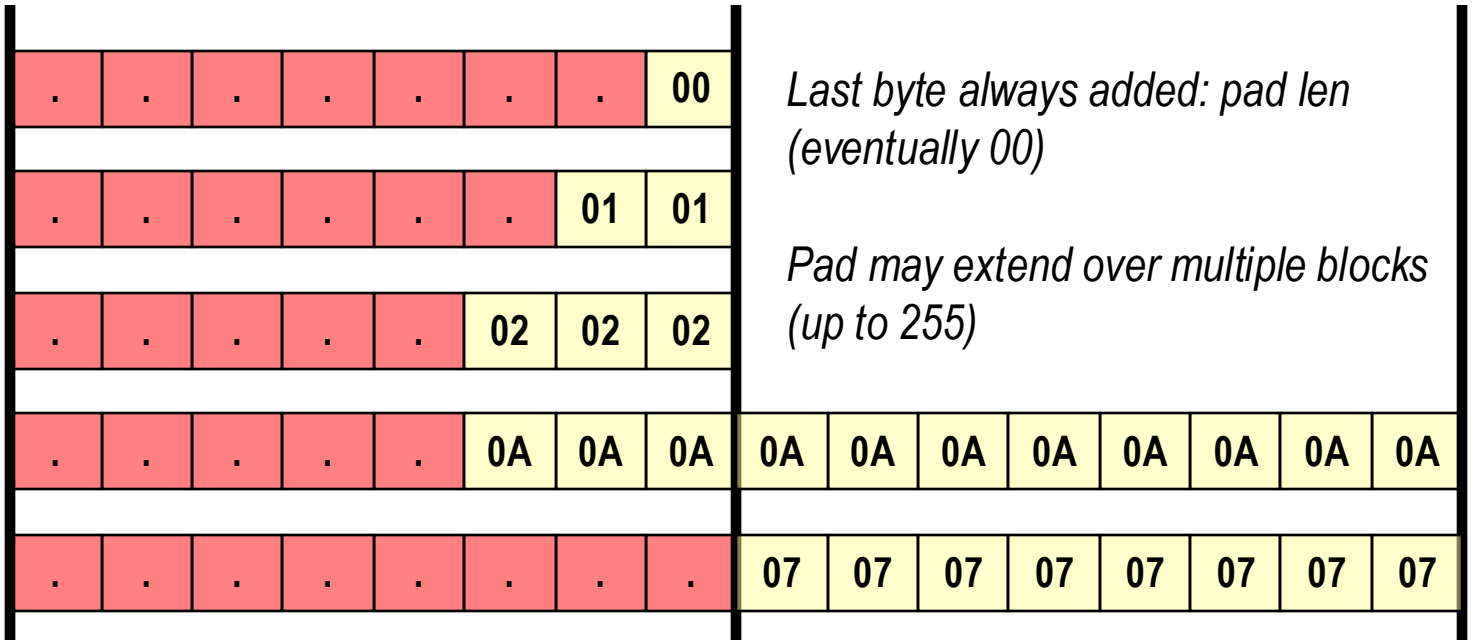
CBC Padding Oracle attack

exploits poor protocol choice (and bad implementation) in TLS 1.0

CBC padding



→TLS padding (say block = 8 B):



CBC decryption steps (TLS 1.0)

➔ **Decrypt**

⇨ *Unless # bytes NOT multiple of block size - return alert message*

→decryption_failed

➔ **Remove padding**

⇨ *Read last byte = pad len L*

⇨ *Remove remaining L last bytes while checking they are equal to L*

⇨ *Invalid pad - return alert message:*

→decryption_failed

➔ **Check MAC**

⇨ *MAC over decrypted msg (no pad) - if fails return alert message:*

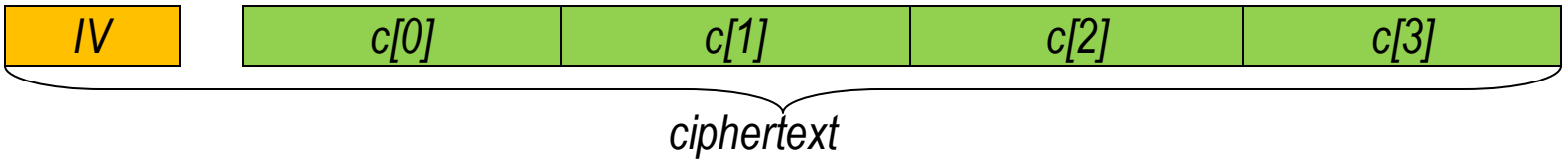
→Bad_record_mac

Typical networking protocols approach: EXPLAIN the error reason

Opposite in crypto: when decryption fails do NOT explain why!!!

(attacker may use such "explanation" to build an attack… see next)

The attack



➔**Goal:**

⇨*decryption of a block, say c[1]*

➔c[0] needs slightly different approach

➔**Attacker ability: Chosen ciphertext attack**

➔**Attacker is able to submit an(other) ciphertext and get result:**

» Decryption failed

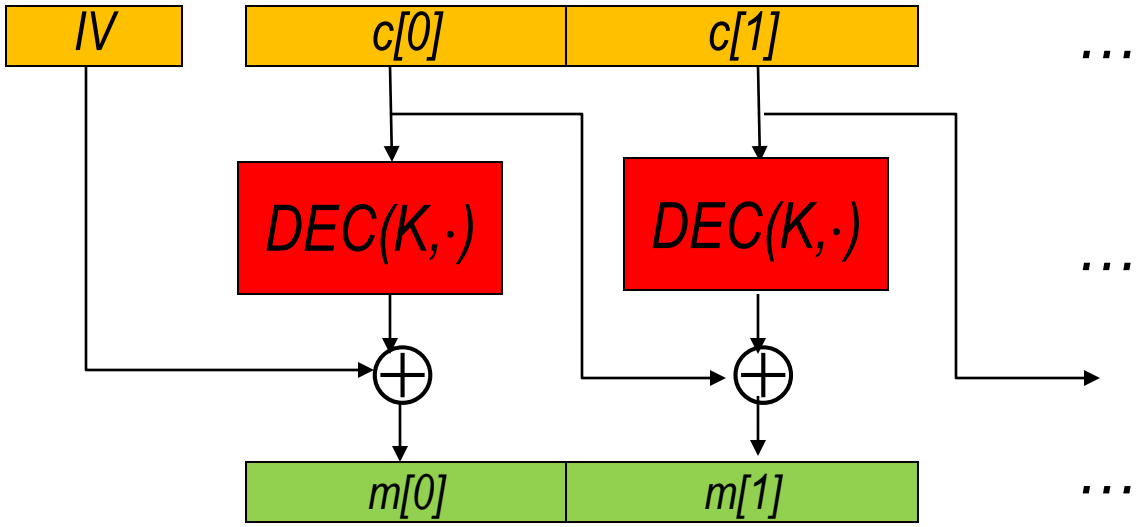
» Bad MAC

➔**Bad MAC: means padding was found OK! We have a "padding oracle"…**

Recall CBC decrypt

➔**c[i] = ENC(K, c[i-1] ⊕ m[i])**

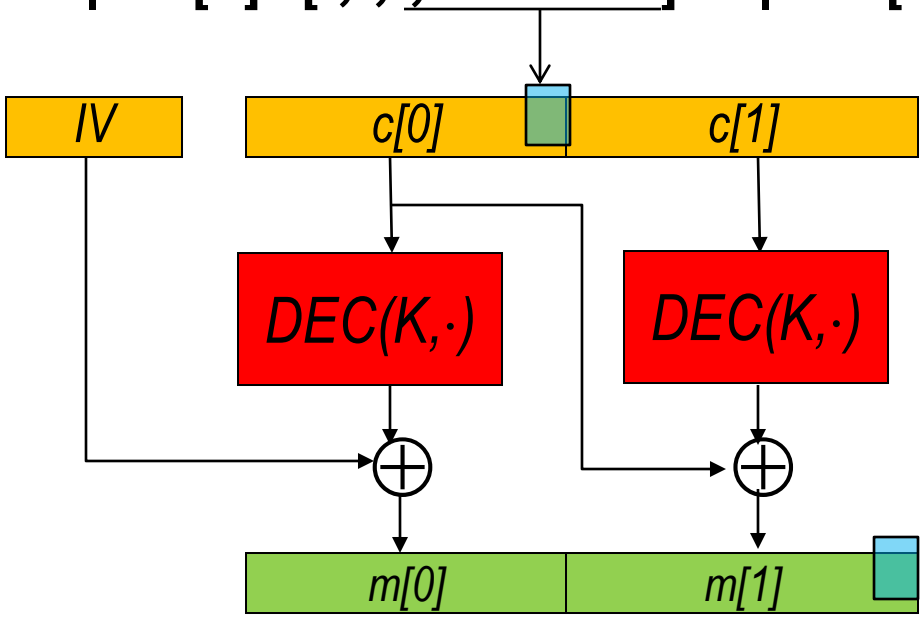
➔**m[i]= c[i-1] ⊕ DEC(K, c[i])**



Start from last byte of m[1]

➔ **Guess: last byte of m[1] = A?**
submit ciphertext (truncated to c[1]):

IV | c[0]⊕[.,.,.,A ⊕0x00] | c[1]

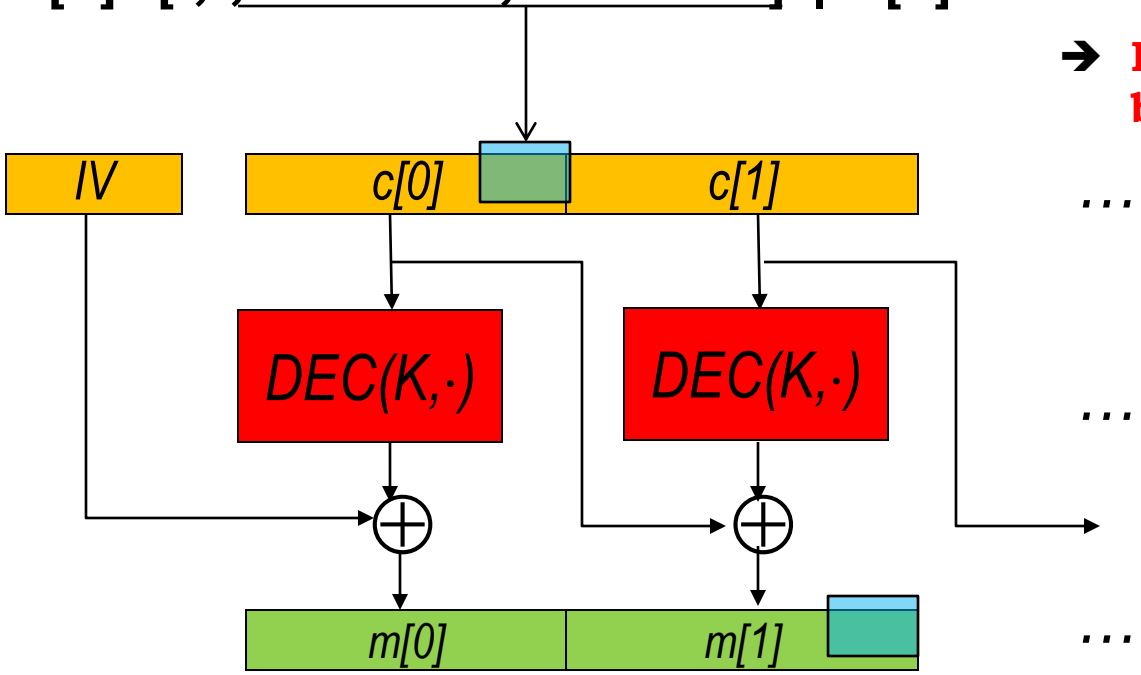


➔ **If guess true, last byte of m[1] = 0x00**
➔ **But then padding would be OK ← bad_record_mac!**
➔ **If guess wrong, ← decryption_failed!!**

Iterate on remaining bytes

➔ **We now know last byte is A**
➔ **Next guess: previous byte of m[1] == F?**
➔ **submit ciphertext**

IV | c[0]⊕[.,., F ⊕0x01, A ⊕0x01] | c[1]

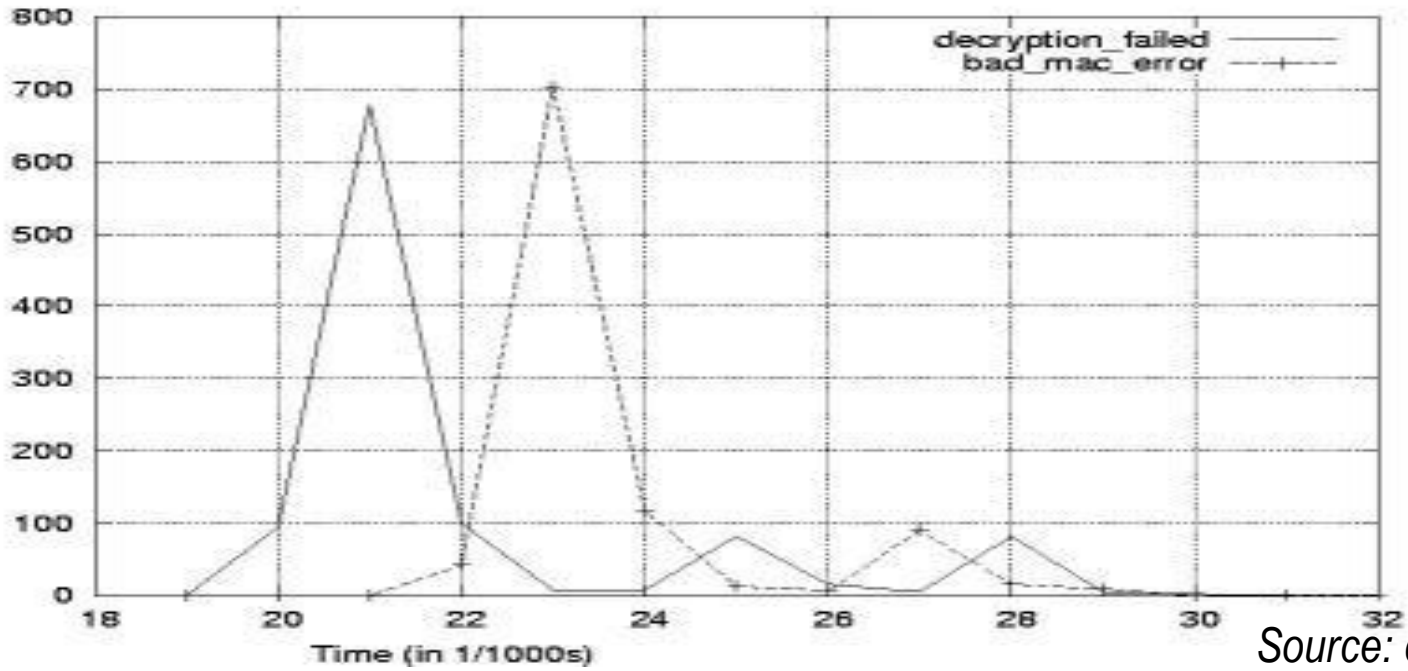


➔ **Example: 8 bytes block:**
- ⇨ *At most 256 guesses for each byte:*
  - ➔ 256 x 8 = 2^{11}
  - ➔ Fast!

➔ **If true, last 2 bytes of m[1] ➔ 0x01!! Padding check OK, error = bad_mac**

Preventing the attack

- **Discovered in 2002 (Vaudenay)**
- **Correct TLS protocol to avoid such "padding oracle"**
  - *Most TLS1.0 implementations: respond with same alert (bad mac) in both cases*
  - *TLS1.1: standardized this*
- **But canvel 2003: side channel "padding oracle"!**
  - *Must correct also* **implementation***!!*
  - *TLS1.1: always perform MAC even if malformed msg*



Source: canvel 2003

Lessons

➔**Encrypt then MAC would NOT have permitted such attack**

➔**When dealing with crypto, we (networkers) need to be more careful in what we report…**

⇨*The least, the better*

➔**Crypto implementations are extremely critical**

⇨*Side channel attacks*

⇨*Don't implement crypto yourself!*

Is the attack practical?

- **Apparently no**
  - *Bad_mac and decryption failed: fatal alerts*
  - *TLS connection aborted*
  - *Next connection will have different key!*
- **But… actually performed on IMAP (Vuagnoux & Canvel, 2003)**
  - *Email IMAP client periodically sends login/passwd every few minutes (1-5)*
  - *Resulting TLS msg plaintext*
    - XXXX LOGIN "username" "password"<cr><nl><MAC><PAD><LEN>
  - *C→ S intercepted with MITM attack (DNS spoofing), and CCA attack performed*
    - Connection abort not an issue
    - Attack (optimized with further dictionary attack strategies) successful in at most few hours

Fixes and follow-ups

➔ **TLS1.2:**

If padding fails, **validate MAC in any case**

➔ **Devil is in the details**

⇨ *But which data is validated? If padding fails, no way to know message size versus padding size*

⇨ *TLS1.2 solution: use the whole data for validation*

→But this is **more data** than a correct message!!

→May require one extra compression function in the HMAC → extra time

➔ **2013, Lucky thirteen, kenny Paterson**

➔ **2014: POODLE**

⇨ *combine Padding Oracle with an SSL3.0 downgrade*

➔ **2015, Lucky microseconds, Albrecht & Paterson**

⇨ *Attack using this very subtle timing channel!!!*

➔ **2016, CVE-2016-2107 (LuckyNegative20)**

⇨ *Attacks the constant-time patch of Lucky13!*

⇨ *https://blog.cloudflare.com/yet-another-padding-oracle-in-openssl-cbc-ciphersuites/*

Fun (?!) Lesson:

➔**Cryptographic Doom Principle**

» © Moxie Marlinspike

⇨*if you have to perform any cryptographic operation before verifying the MAC on a message you've received, it will somehow inevitably lead to doom.*

➔**Encrypt then MAC would NOT have permitted such a painful Pandora's Box!!**

⇨*A 15 years nightmare solved only by… removing the origin problem!*

➔AEAD mandatory in TLS1.3